

Experiment No.	7
Experiment Title	Dimensionality Reduction and Model Evaluation (With and Without PCA) ¹
Student Name	Isha Venkataraman
Register Number	3122247001023
Faculty	Dr. Poreddy Ajay Kumar Reddy
Submission Date	

1 Objective

To study the effect of dimensionality reduction using Principal Component Analysis (PCA) on the performance of ten different machine learning classifiers. Each model is trained and validated both on the original feature space (No-PCA) and on a PCA-reduced feature space (With-PCA), using hyperparameter tuning and 5-fold cross-validation, so that the two settings can be compared fairly.

2 Problem Statement

Given a dataset of labelled samples, the task is to build classifiers that can correctly predict the class of a new sample. The input is a set of numeric features describing each sample and the output is the predicted class label. The experiment compares model performance when the classifiers are trained directly on the standardised features against training them on a reduced set of principal components, to see whether PCA improves accuracy, stabilises cross-validation performance, or mainly helps with training time.

As an extension, the same No-PCA / With-PCA workflow is repeated on a regression problem using the regressor version of each of the ten algorithms, to check whether the observations made for classification also hold for a continuous target.

3 Dataset Description

Digits Dataset (Classification)

Dataset Name	Digits (handwritten digit images)
Dataset Source	UCI ML Repository, bundled in scikit-learn (<code>sklearn.datasets.load_digits</code>)
Number of Samples	1797
Number of Features	64 (8x8 grayscale pixel intensities, values 0–16)
Number of Classes	10 (digits 0–9)
Missing Values	0 missing values, 0 duplicate rows
Train-Test Split	80/20, stratified on the digit label (1437 train / 360 test)

Diabetes Dataset (Regression Extension)

Dataset Name	Diabetes (physiological / blood-panel measurements)
Dataset Source	UCI-derived, bundled in scikit-learn (<code>sklearn.datasets.load_diabetes</code>)
Number of Samples	442
Number of Features	10 (age, sex, bmi, bp, s1–s6 blood serum measurements)
Target	Continuous disease-progression score, range 25 to 346
Missing Values	0 missing values, 0 duplicate rows
Train-Test Split	80/20 (353 train / 89 test)

4 Exploratory Data Analysis

Digits Dataset

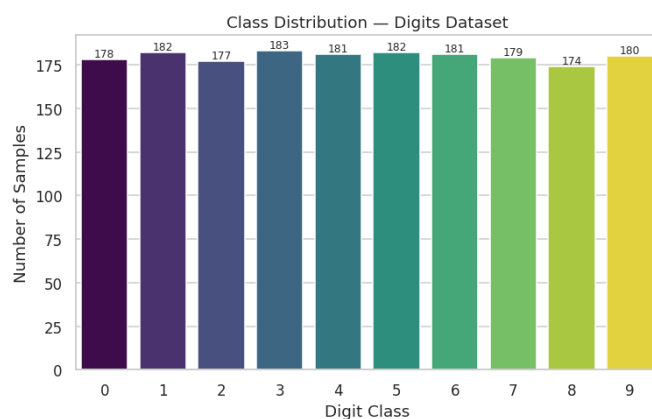


Figure 1: Class distribution across the 10 digit classes.

Inference: The 10 digit classes are close to evenly distributed, with roughly 175–183 samples per class. Because there is no class imbalance, accuracy is a reasonable metric to use alongside macro F1 for this dataset. This also means cross-validation folds are unlikely to be skewed toward any one digit.

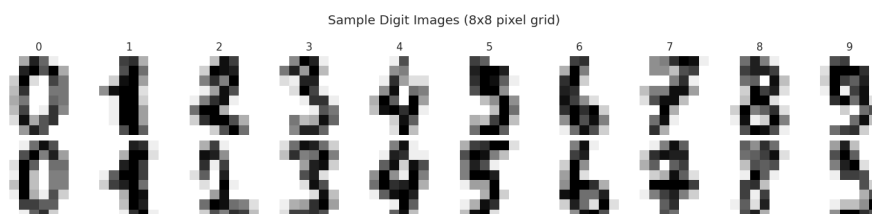


Figure 2: Sample 8x8 pixel images for each digit class.

Inference: Each digit is represented as an 8x8 grid of grayscale pixel intensities, giving 64 raw features per sample. Some digits such as 3, 5, 8 and 9 share similar stroke

patterns, which is one reason classifiers can confuse them. This visual overlap is also what makes the dataset a good candidate for PCA, since neighbouring pixels tend to vary together.

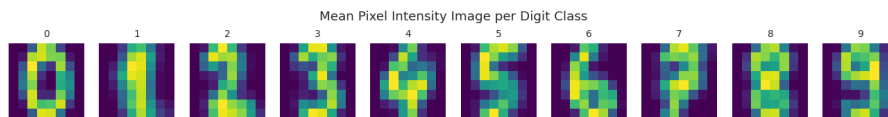


Figure 3: Mean pixel-intensity image per digit class.

Inference: Averaging all samples of a class produces a recognisable prototype for each digit, which shows the classes are visually well separated on average. The border pixels stay dark (low intensity) for every class since digits are centred in the frame, so those pixels contribute very little discriminative information. This partly explains why a large share of the 64 features can be discarded with limited loss of information.

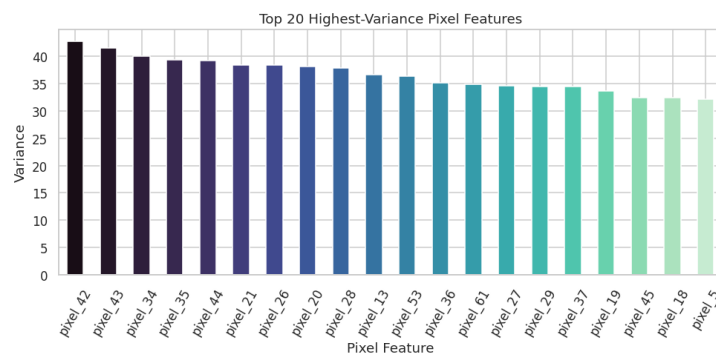


Figure 4: Twenty highest-variance pixel features.

Inference: Variance is concentrated in a subset of pixels near the centre of the image, while border pixels have almost zero variance across the dataset. Features with near-zero variance carry little information for separating classes and are natural candidates to be compressed by PCA. This uneven spread of variance is a first indication that the 64-dimensional space is redundant.

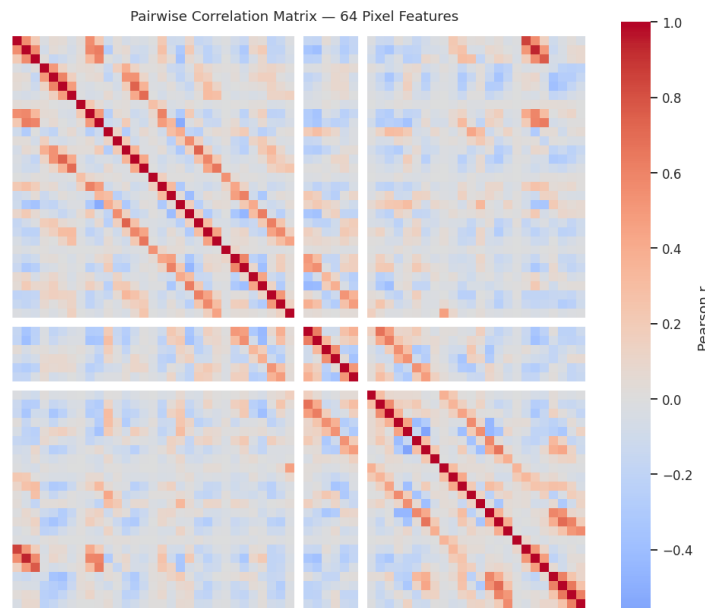


Figure 5: Pairwise correlation matrix of the 64 pixel features.

Inference: Neighbouring pixels show visibly high correlation, appearing as bright blocks along the diagonal region of the heatmap. This confirms that many of the 64 features carry overlapping information rather than independent signal. High correlation between features is exactly the condition under which PCA is expected to be useful, since it can combine correlated pixels into fewer, less redundant components.

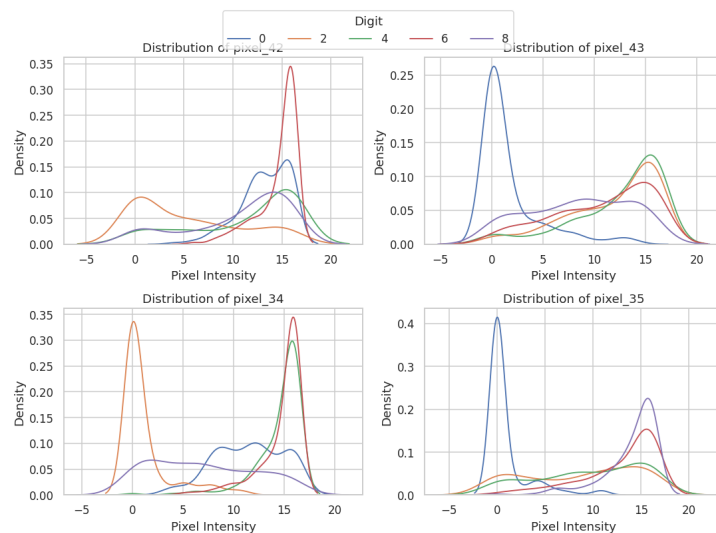


Figure 6: Distribution of the four highest-variance pixel features, split by digit class.

Inference: The intensity distributions for these high-variance pixels differ noticeably across digit classes, which is why these particular pixels are informative for classification. Some classes overlap heavily on a single pixel but separate more clearly when several pixels are considered together. This supports using a multivariate technique such as PCA rather than judging individual pixels in isolation.

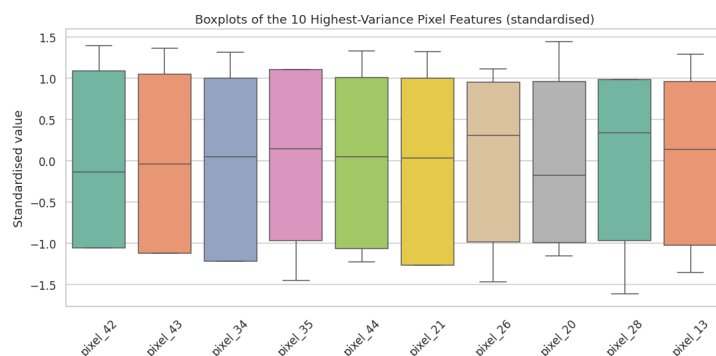


Figure 7: Boxplots of the 10 highest-variance pixel features after standardisation.

Inference: After standardisation, most features are centred close to zero with a modest number of points beyond the whiskers, but nothing resembling extreme outliers. Because pixel intensities are bounded between 0 and 16, there are no unbounded outlier values to worry about. This means no special outlier removal step was needed beyond the usual standardisation.

Diabetes Dataset (Extension)

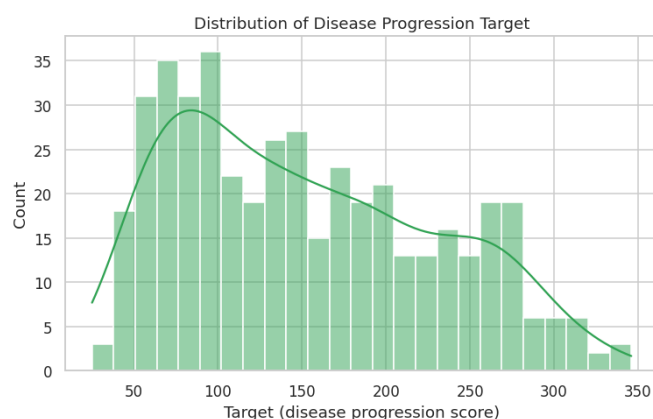


Figure 8: Distribution of the disease-progression target.

Inference: The target is roughly bell-shaped with a slight right skew, and most values fall between 75 and 250. There is no obvious floor or ceiling effect that would distort a regression fit. This distribution is fairly typical for a health-outcome score and does not require a log transform.

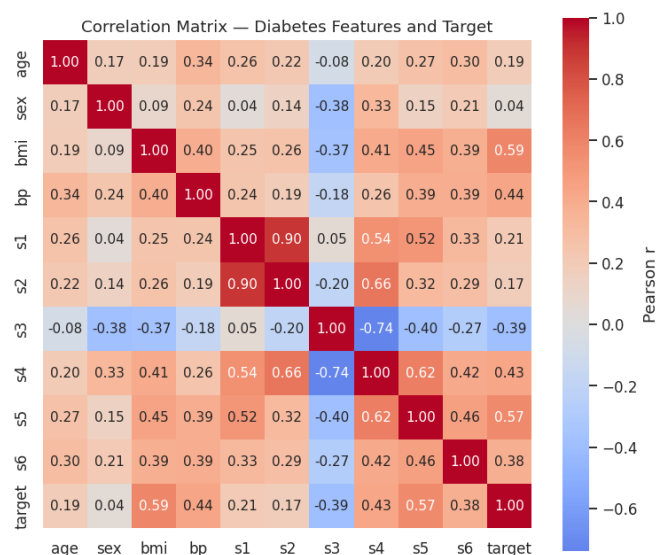


Figure 9: Correlation matrix of the 10 Diabetes features and the target.

Inference: bmi and s5 show the strongest positive correlation with the target, while some of the blood-serum features (s1, s2, s4) are strongly correlated with each other. This overlap among the s-series features is the main source of redundancy PCA can exploit here, even though the feature count is already small. Because the dataset only has 10 features to begin with, the amount of redundancy – and hence the benefit from PCA – is much smaller than in the Digits case.

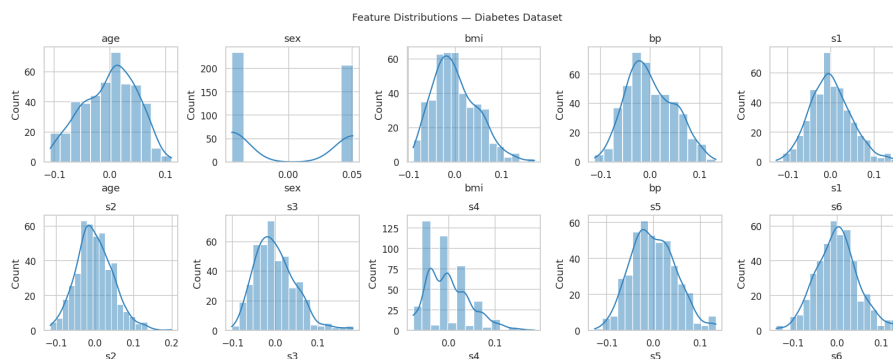


Figure 10: Distribution of each of the 10 Diabetes features.

Inference: All 10 features are mean-centred, since scikit-learn ships this dataset pre-standardised, and most look approximately symmetric. `sex` is binary-coded and appears as a two-spike distribution rather than a continuous one. No extreme skew is present, so the standard scaling applied afterwards mainly rescales rather than reshapes these distributions.

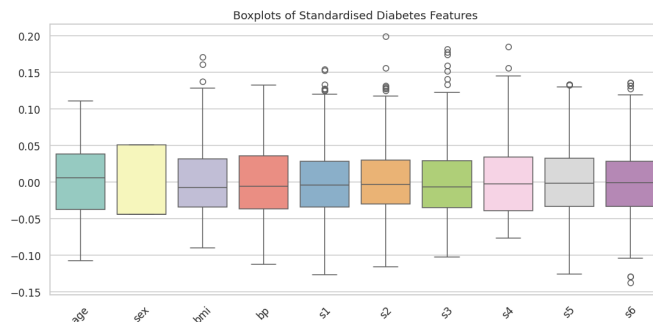


Figure 11: Boxplots of the 10 standardised Diabetes features.

Inference: A handful of points sit beyond the whiskers for several features, which is expected in a health dataset of this size and is not severe enough to require trimming. The spread is broadly comparable across features, so no single variable dominates a distance-based model purely because of scale. This confirms standardisation alone is sufficient preprocessing here.

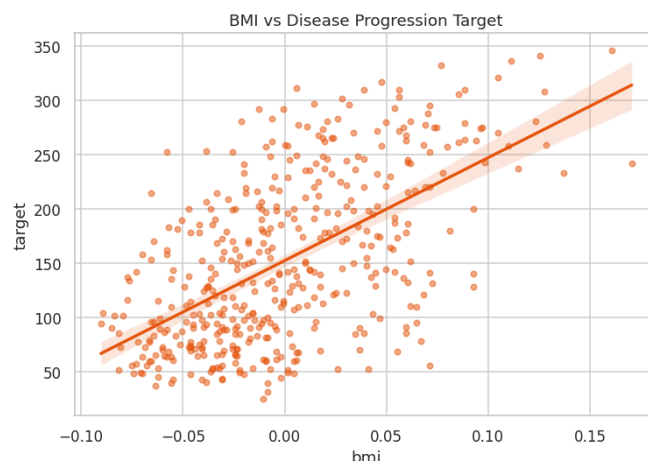


Figure 12: BMI plotted against the disease-progression target.

Inference: There is a clear positive linear trend between BMI and the target, consistent with BMI being the strongest single predictor in the correlation matrix. The relationship is noisy rather than a tight line, which is why no single feature by itself will produce a high R^2 score. This scatter is a useful sanity check before fitting any regression model.

5 Data Preprocessing

- **Missing values:** none found in either dataset, so no imputation was required.
- **Encoding:** not applicable – all features in both datasets are already numeric.
- **Feature scaling:** `StandardScaler` was fit on the training split only and then applied to the test split, to avoid data leakage into the test set.
- **Train-test split:** 80/20, stratified on the label for the classification dataset (Digits) and a plain 80/20 split for the regression dataset (Diabetes).

- **Dimensionality reduction:** PCA was fit on the standardised training features only, using a 95% cumulative-explained-variance target, and then applied to the test split using the same fitted transform.

6 Machine Learning Algorithms

Ten classifiers were evaluated. A short summary of each follows.

Support Vector Machine (SVM)

Working principle: Finds the hyperplane that maximises the margin between classes, using a kernel (linear or RBF here) to handle non-linear boundaries.

Advantages: Effective in high-dimensional spaces and robust to overfitting when the margin is well chosen.

Limitations: Training cost grows quickly with dataset size and results are sensitive to the choice of kernel and regularisation parameter C .

Naive Bayes

Working principle: Applies Bayes' theorem assuming features are conditionally independent given the class, using a Gaussian distribution per feature in this experiment.

Advantages: Very fast to train, works reasonably well even with limited data.

Limitations: The independence assumption rarely holds exactly, which hurts accuracy when features are correlated, as pixel features are here.

k-Nearest Neighbors (KNN)

Working principle: Classifies a point by majority vote among its k closest training points, using a chosen distance metric.

Advantages: Simple, no training phase, adapts naturally to complex decision boundaries.

Limitations: Prediction is slow on large datasets and performance degrades in high-dimensional spaces where distances become less meaningful.

Logistic Regression

Working principle: Models the log-odds of each class as a linear combination of the features, fit by maximum likelihood.

Advantages: Fast, interpretable, and a strong baseline for linearly separable problems.

Limitations: Struggles with non-linear decision boundaries unless features are transformed first.

Decision Tree

Working principle: Recursively splits the feature space using the feature/threshold that best separates classes at each node.

Advantages: Easy to interpret and requires little preprocessing.

Limitations: Prone to overfitting if depth is not controlled, and small changes in data can produce very different trees.

Random Forest

Working principle: An ensemble of decision trees trained on bootstrapped samples and random feature subsets, combined by majority vote.

Advantages: Reduces the overfitting of a single tree and handles high-dimensional data well.

Limitations: Less interpretable than a single tree and can be slower to train with many estimators.

AdaBoost

Working principle: Trains a sequence of weak learners, each focusing more on the samples the previous ones misclassified, and combines them with weighted voting.

Advantages: Can turn weak learners into a strong classifier and needs little parameter tuning.

Limitations: Sensitive to noisy data and outliers, since misclassified noisy points keep receiving more weight.

Gradient Boosting

Working principle: Builds trees sequentially, where each new tree fits the residual errors of the current ensemble.

Advantages: Usually achieves high accuracy and handles mixed feature types well.

Limitations: Training is sequential and slower than bagging methods, and it can overfit if the number of estimators is too high.

XGBoost

Working principle: A regularised, optimised implementation of gradient boosting with additional techniques for speed and to control overfitting.

Advantages: Fast, accurate, and widely used in practice for tabular data.

Limitations: Has more hyperparameters to tune than plain gradient boosting, and the extra speed comes from added implementation complexity.

Stacking

Working principle: Combines predictions from several base learners (here Decision Tree, KNN and SVM) using a meta-learner (Logistic Regression) trained on their outputs.

Advantages: Can outperform any individual base model by learning how to best combine their strengths.

Limitations: More expensive to train and tune than a single model, and the benefit depends heavily on how diverse the base learners are.

7 Experimental Setup

Python Version	3.x (Anaconda / Jupyter environment)
Scikit-Learn Version	1.x
Additional Library	XGBoost
IDE	Jupyter Notebook
Operating System	Not recorded by the notebook
Hardware	Standard CPU (no GPU used)

8 Hyperparameter Selection

PCA was applied with a 95% cumulative explained variance target. Table 3 summarises the choice for the Digits dataset.

Table 3: PCA Variance Explained (Digits)

Setting	Chosen Components / Variance Target	Explained Variance (%)	Justification
With-PCA	95% cumulative variance (40 of 64 components)	95.16	95% is a conventional threshold that keeps almost all discriminative signal while discarding low-variance, noise-dominated pixel components; it gives a large reduction from 64 to 40 dimensions.

Table 4: SVM – Hyperparameter Tuning Results

Kernel	C Values Tried	Gamma Values Tried	Perf. (No-PCA)	Perf. (With-PCA)
rbf, linear	0.1, 1, 10	scale, 0.01	0.9805	0.9805

Table 5: Naïve Bayes – Smoothing Choices

Smoothing Parameter	Performance (No-PCA)	Performance (With-PCA)
1e-9, 1e-8, 1e-7, 1e-6	0.8302	0.8901

Table 6: KNN – Hyperparameter Tuning

k Values	Weights	Distance Metrics	Perf. (No-PCA)	Perf. (With-PCA)
3, 5, 7, 9	uniform, distance	euclidean, manhattan	0.9777	0.9750

Logistic Regression, Decision Tree, Random Forest, AdaBoost, Gradient Boosting, XGBoost and Stacking were tuned the same way; their best parameters and mean 5-fold accuracy are summarised below instead of one table per model.

Model	Best Params (No-PCA)	Best Params (With-PCA)	Avg Acc. (No-PCA)	Avg Acc. (With-PCA)
Logistic Regression	C=1, solver=lbfgs	C=1, solver=lbfgs	0.9694	0.9694
Decision Tree	max_depth=10, min_samples_split=2	max_depth=None, min_samples_split=2	0.8448	0.8448
Random Forest	max_depth=10, n_estimators=200	max_depth=20, n_estimators=100	0.9770	0.9770
AdaBoost	learning_rate=1.0, n_estimators=100	learning_rate=1.0, n_estimators=100	0.8017	0.8017
Gradient Boosting	learning_rate=0.1, max_depth=3, n_estimators=150	learning_rate=0.1, max_depth=3, n_estimators=100	0.9652	0.9652
XGBoost	learning_rate=0.1, max_depth=5, n_estimators=150	learning_rate=0.1, max_depth=3, n_estimators=150	0.9659	0.9659
Stacking	final_estimator__C=0.1	final_estimator__C=1	0.9882	0.9882

Cross-Validation Results (Digits, All Models)

Each model was tuned with `GridSearchCV` and then evaluated with 5-fold stratified cross-validation, in both the original feature space (No-PCA) and the PCA-reduced space (With-PCA).

Table 8: 5-Fold Accuracy, No-PCA

Model	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5	Avg
SVM	0.9861	0.9757	0.9861	0.9721	0.9826	0.9805
Naive Bayes	0.8160	0.8507	0.8397	0.8084	0.8362	0.8302
KNN	0.9757	0.9896	0.9861	0.9617	0.9756	0.9777
Logistic Regression	0.9583	0.9722	0.9721	0.9652	0.9791	0.9694
Decision Tree	0.8472	0.8125	0.8571	0.8502	0.8571	0.8448
Random Forest	0.9722	0.9826	0.9791	0.9652	0.9861	0.9770
AdaBoost	0.8090	0.7986	0.7909	0.8397	0.7700	0.8017
Gradient Boosting	0.9514	0.9792	0.9686	0.9582	0.9686	0.9652
XGBoost	0.9583	0.9688	0.9582	0.9617	0.9826	0.9659
Stacking	0.9861	0.9861	0.9895	0.9895	0.9895	0.9882

Table 9: 5-Fold Accuracy, With-PCA

Model	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5	Avg
SVM	0.9896	0.9757	0.9861	0.9686	0.9826	0.9805
Naive Bayes	0.8715	0.9028	0.9094	0.8502	0.9164	0.8901
KNN	0.9722	0.9722	0.9826	0.9721	0.9756	0.9750
Logistic Regression	0.9583	0.9688	0.9686	0.9477	0.9721	0.9631
Decision Tree	0.8056	0.8125	0.8641	0.8014	0.8118	0.8191
Random Forest	0.9549	0.9722	0.9756	0.9443	0.9686	0.9631
AdaBoost	0.7500	0.7639	0.7596	0.7387	0.7909	0.7606
Gradient Boosting	0.9062	0.9410	0.9547	0.9199	0.9582	0.9360
XGBoost	0.9271	0.9479	0.9617	0.9233	0.9721	0.9464
Stacking	0.9826	0.9896	0.9861	0.9721	0.9861	0.9833

9 Results

Stacking gave the highest mean cross-validation accuracy in both settings and was selected as the final model. Its performance on the held-out test set (With-PCA) is summarised below.

Metric	Value
Accuracy	0.9806
Macro F1-score	0.9804
Macro-average ROC-AUC	1.000

10 Result Visualization

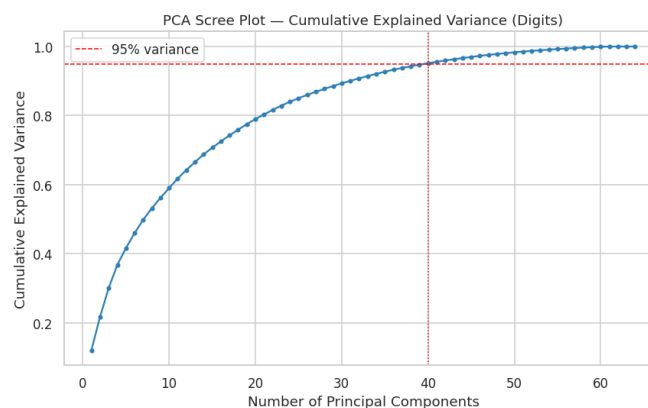


Figure 13: PCA scree plot – cumulative explained variance (Digits).

Inference: The curve rises steeply for the first 10–15 components and then flattens, showing that most of the variance is captured by a fraction of the original 64 dimensions. 40 components are needed to reach the 95% variance target used in this experiment. This confirms the earlier correlation heatmap’s suggestion that the pixel features carry a lot of redundant information.

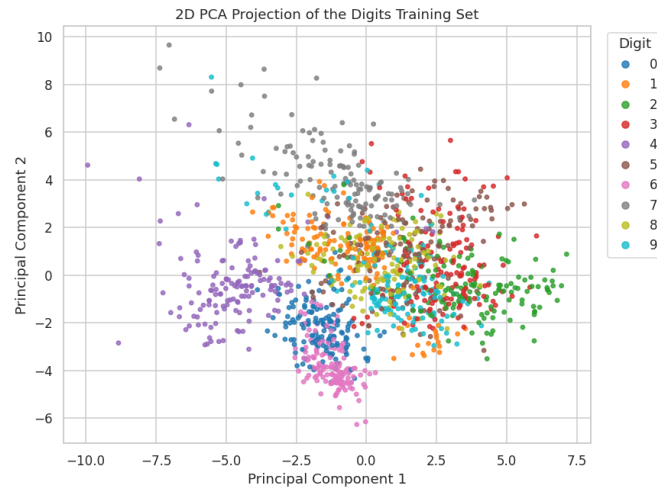


Figure 14: 2D PCA projection of the training set.

Inference: Even with only the first two components, several digit classes (e.g. 0, 4, 6) form fairly distinct clusters, while others overlap more, such as 3, 5, 8 and 9. This matches the visual similarity noted earlier in the sample digit images. It shows that a good amount of class-relevant structure already survives after reducing to just two dimensions.

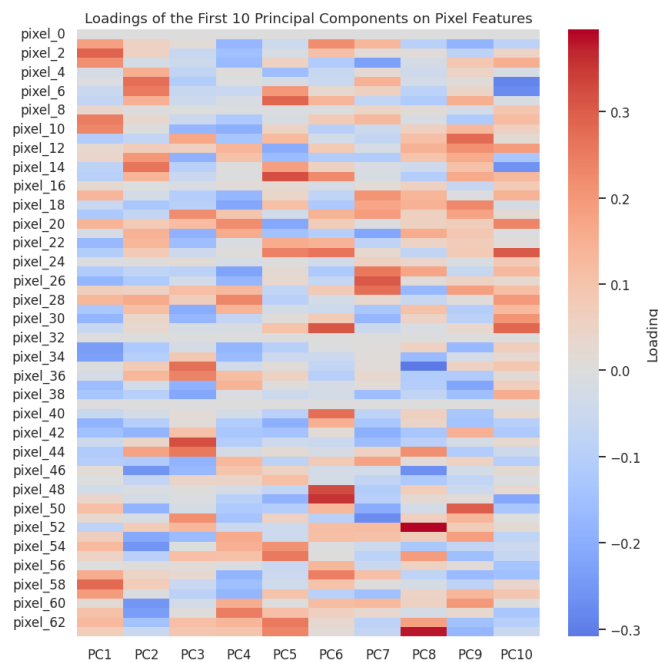


Figure 15: Loadings of the first 10 principal components on the pixel features.

Inference: Each component is a different weighted combination of the original 64 pixels, with some components loading heavily on the centre of the image and others on the edges. No single original pixel dominates any one component, which is expected since PCA mixes correlated features together. This heatmap is mainly useful for understanding what each component represents rather than for direct classification.

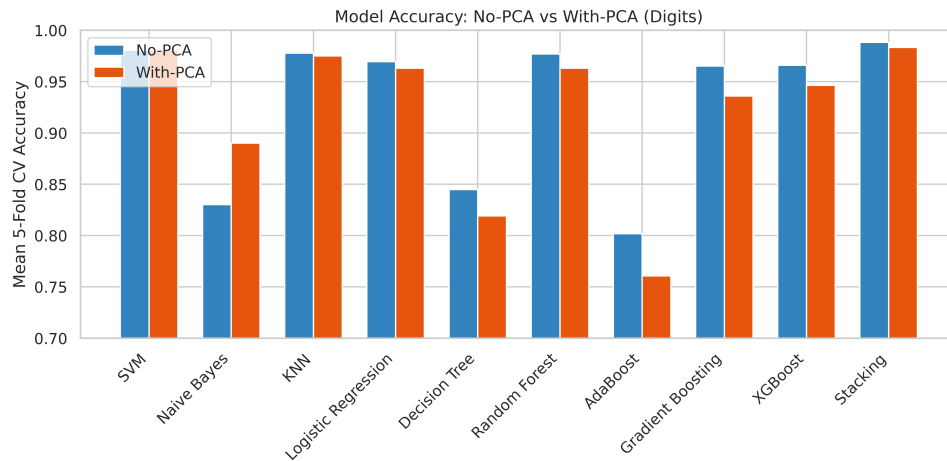


Figure 16: Mean 5-fold CV accuracy for all 10 models, No-PCA vs With-PCA.

Inference: SVM, KNN and Stacking are the strongest models in both settings, staying above 97% accuracy. Naive Bayes is the only model to clearly improve with PCA, since removing correlated pixels makes its independence assumption less badly violated. Tree-based ensembles (Random Forest, Gradient Boosting, XGBoost, AdaBoost) perform slightly worse with PCA, since they already handle correlated raw features reasonably well through their own splitting process.

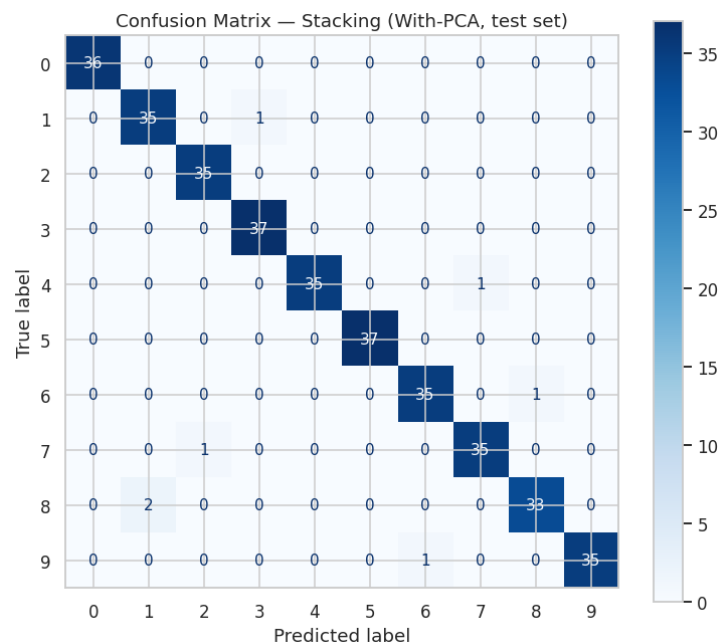


Figure 17: Confusion matrix for the best model (Stacking, With-PCA, test set).

Inference: Almost all predictions fall on the diagonal, matching the 98.06% test accuracy. The few errors that do occur are between visually similar digits, which is consistent with the overlap seen earlier in the 2D PCA scatter plot. No single class stands out as a major failure point for this model.

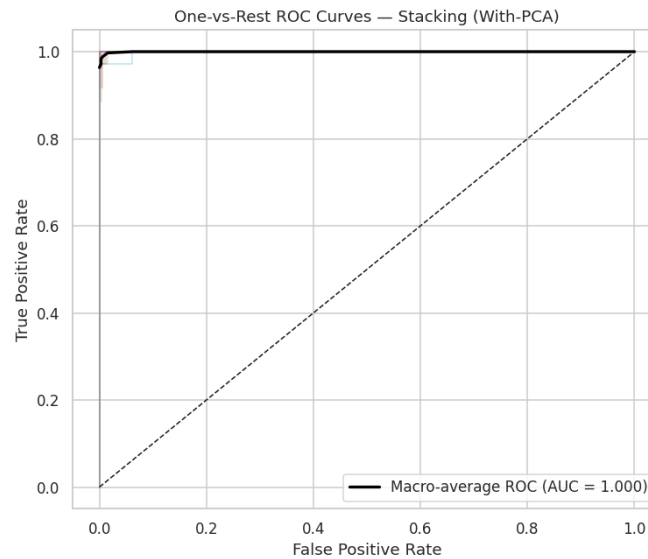


Figure 18: One-vs-rest ROC curves for the best model.

Inference: Each per-class curve rises almost immediately to the top-left corner, and the macro-average ROC-AUC is 1.000. This indicates the model separates each digit from the rest of the classes almost perfectly on the test set. Such a high AUC alongside a 98% accuracy suggests the errors that do occur are concentrated in a small number of hard cases rather than a systematic weakness.

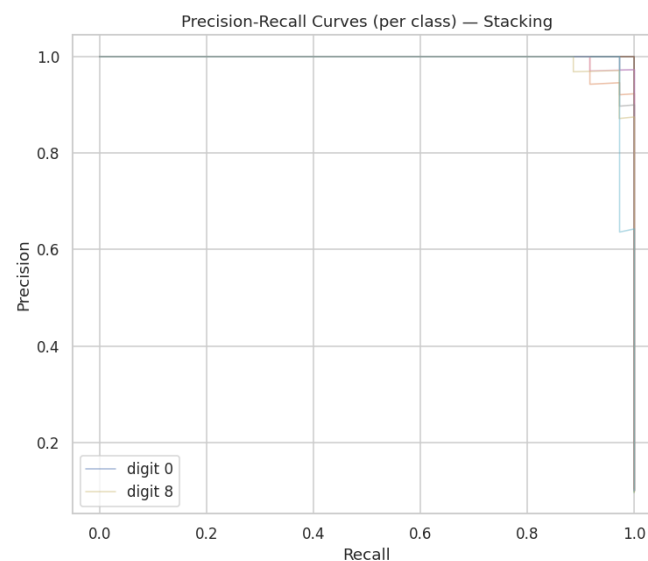


Figure 19: Precision-recall curves (per class, digits 0 and 8 highlighted).

Inference: Precision stays high across almost the full recall range for both highlighted classes, meaning the model rarely trades away precision to catch more true positives. Digit 8, which is visually similar to several other digits, still holds a strong curve, though not quite as tight as digit 0. This is a useful secondary check alongside the confusion matrix since precision-recall curves are less optimistic than ROC curves for imbalanced error costs.

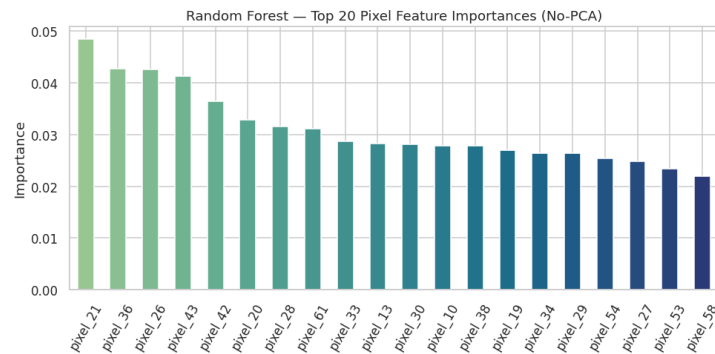


Figure 20: Top 20 pixel feature importances from Random Forest (No-PCA).

Inference: The most important pixels sit in the centre of the 8x8 grid, matching the region highlighted earlier as high variance. Border pixels do not appear in the top 20, confirming they add little value to the model. This ranking is only meaningful in the original pixel space, which is why it is computed from the No-PCA Random Forest rather than the PCA version.

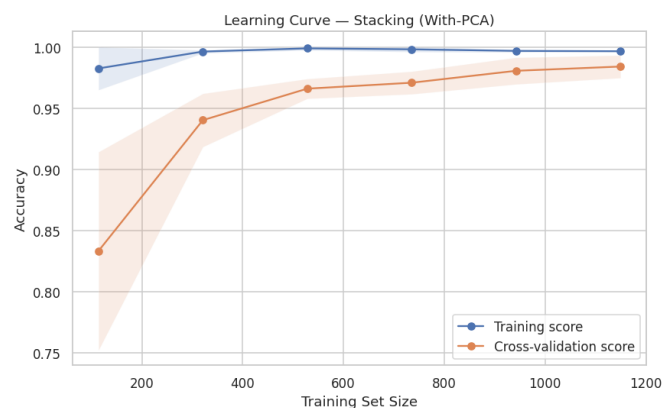


Figure 21: Learning curve for the best model (Stacking, With-PCA).

Inference: Training and cross-validation scores converge closely as the training set size grows, with only a small gap remaining at the full training size. This indicates the model is not overfitting and would likely benefit only marginally from more training data. The validation curve rises quickly even with a fraction of the data, showing the task is learnable with a moderate sample size.

Regression Extension – Result Visualization (Diabetes)

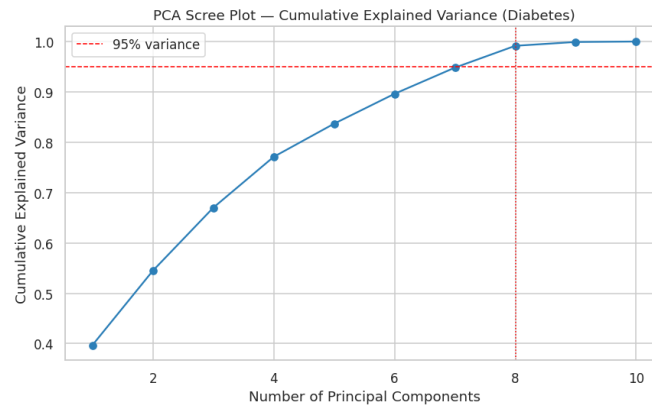


Figure 22: PCA scree plot – cumulative explained variance (Diabetes).

Inference: The curve rises quickly and 8 of the 10 original components are needed to reach 95% variance, showing far less redundancy than in the Digits dataset. This matches the smaller, less repetitive nature of a 10-feature clinical dataset. PCA therefore has much less room to compress this dataset than it did for Digits.

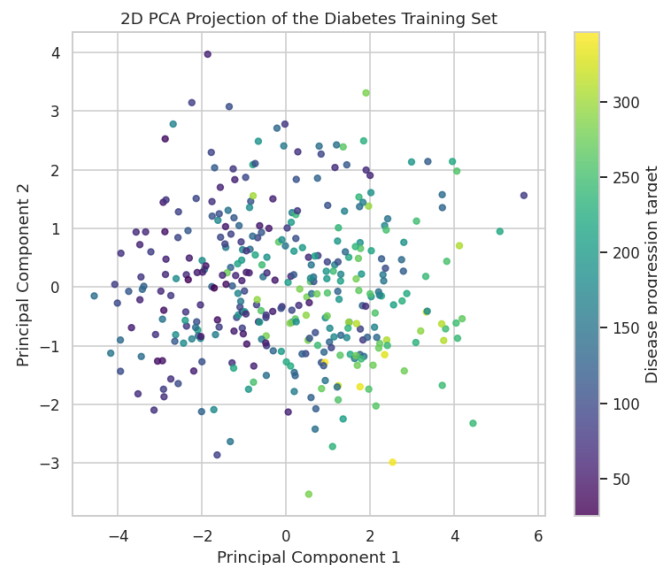


Figure 23: 2D PCA projection of the Diabetes training set, coloured by target value.

Inference: There is a rough gradient of target values across the 2D projection, with higher scores trending toward one side of the plot, but the separation is much fuzzier than the class clusters seen for Digits. This is expected since regression targets are continuous and the original feature set is already compact. It suggests the first two components alone will not be sufficient to build an accurate regressor.



Figure 24: Mean 5-fold CV R^2 for all 10 regressors, No-PCA vs With-PCA.

Inference: Linear Regression, Bayesian Ridge and SVR give the best R^2 scores (around 0.47–0.48) in both settings, while Decision Tree performs noticeably worse, especially with PCA. Differences between No-PCA and With-PCA are small for most models, confirming that PCA has a limited effect on this already low-dimensional dataset. Ensemble methods again show little change either way, consistent with the classification results.

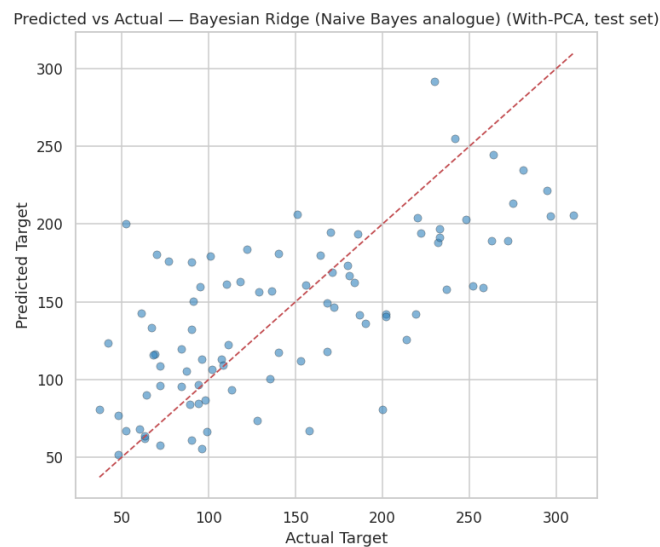


Figure 25: Predicted vs. actual target values for the best regressor (Bayesian Ridge, With-PCA, test set).

Inference: Points scatter around the diagonal reference line with a moderate spread, matching a test R^2 of 0.4581. There is no obvious systematic bias such as consistent over- or under-prediction at either end of the target range. The spread reflects the inherent difficulty of predicting disease progression from only 10 baseline measurements.

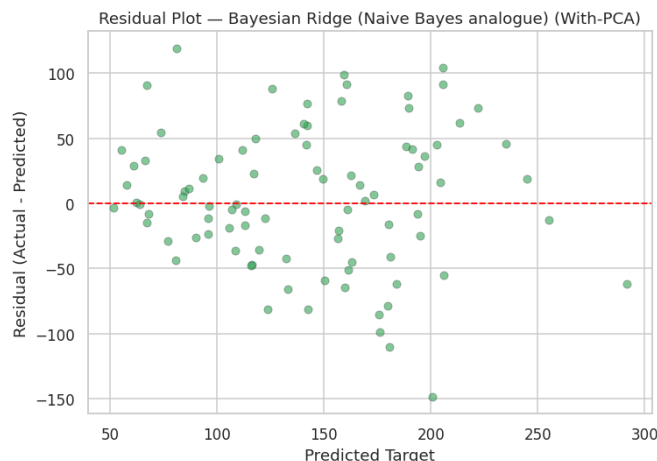


Figure 26: Residual plot for the best regressor.

Inference: Residuals are scattered fairly randomly around zero without a clear funnel or curve shape, suggesting the linear-style model is a reasonable fit and errors are not strongly heteroscedastic. A few larger residuals appear at higher predicted values, but not enough to indicate a systematic pattern. This supports using a linear model such as Bayesian Ridge as the final choice for this dataset.

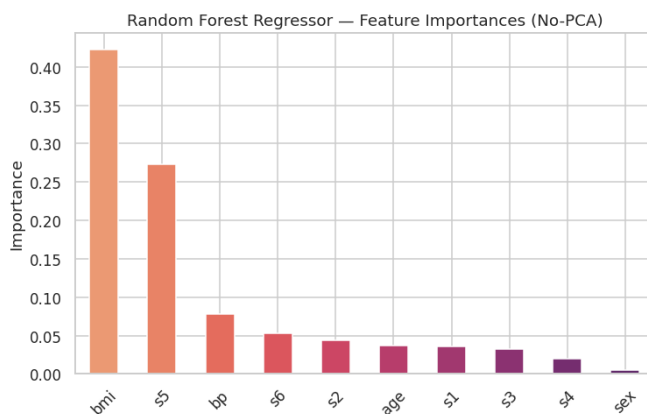


Figure 27: Random Forest feature importances (No-PCA, Diabetes).

Inference: bmi and s5 stand out as the two most important features, consistent with their strong correlation with the target seen earlier. The remaining features contribute more modestly and fairly evenly. This lines up with established clinical understanding that BMI and certain blood-serum markers are strong predictors of disease progression.

11 Discussion

Across both the classification and regression experiments, PCA is most useful when the original feature space is large and internally correlated. On the 64-feature Digits dataset, PCA reduced dimensionality by more than a third (to 40 components) while keeping 95% of the variance, and it gave a real improvement to Naive Bayes and a mild stability benefit to distance- and margin-based models such as KNN, SVM and Logistic Regression. Tree-based ensembles (Random Forest, AdaBoost, Gradient Boosting, XGBoost) were largely

unaffected or slightly worse with PCA, since they already perform their own implicit feature selection at each split and lose some of the direct interpretability of pixel-level splits once features are rotated into components.

On the smaller, already compact 10-feature Diabetes dataset, PCA's effect was much smaller across the board, since there is less redundancy left to remove. The overall pattern – PCA helping linear and instance-based models more than tree-based ensembles – held in both experiments, suggesting it is a reasonably general observation rather than something specific to one dataset.

A limitation of this study is that only one variance threshold (95%) was tested for PCA; sweeping this value, or comparing against other reduction methods such as t-SNE or feature selection, could give a fuller picture. Compute time for hyperparameter tuning was also considerable for the Digits experiment (about 25 minutes across all 10 models), which is a practical cost of running an exhaustive grid search on 10 models in two feature spaces.

12 Conclusion

PCA is a useful default when working with a wide, correlated feature space such as the 64-pixel Digits dataset, where it gave a large dimensionality reduction with little to no accuracy loss and a clear improvement for Naive Bayes. Its benefit shrinks close to zero on a compact, already-informative feature set such as Diabetes. Tree-based ensembles were largely PCA-agnostic in both experiments, and Stacking, built from a mix of base learners, tracked the behaviour of its most PCA-sensitive base learner. Overall, PCA should be applied selectively and validated with cross-validation, as done here, rather than assumed to help in every setting.

13 References

1. F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
2. T. Chen and C. Guestrin, “XGBoost: A Scalable Tree Boosting System,” in *Proc. 22nd ACM SIGKDD Int. Conf. on Knowledge Discovery and Data Mining*, 2016, pp. 785–794.
3. I. T. Jolliffe, *Principal Component Analysis*, 2nd ed. New York, NY, USA: Springer, 2002.
4. UCI Machine Learning Repository, *Optical Recognition of Handwritten Digits Data Set*. [Online]. Available: <https://archive.ics.uci.edu/>